Regression Models using Scikit-learn

This notebook demonstrates regression analysis using a Kaggle dataset: House Prices (Advanced Regression Techniques). Models implemented: Linear Regression, Ridge Regression, and Lasso Regression.

Step 1: Import Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.preprocessing import StandardScaler
```

Step 2: Load Kaggle Dataset

Download the dataset from Kaggle: House Prices - Advanced Regression Techniques. Place `train.csv` in the same folder as this notebook.

```
df = pd.read_csv('train.csv')
df.head()
```

	Id	MSSubClass	MSZoning	LotFrontage	LotArea	Street	Alley	LotShape	LandContour	Utilities	...	PoolArea	PoolQC	Fence
0	1	60	RL	65.0	8450	Pave	NaN	Reg	Lvl	AllPub	...	0	NaN	NaN
1	2	20	RL	80.0	9600	Pave	NaN	Reg	Lvl	AllPub	...	0	NaN	NaN
2	3	60	RL	68.0	11250	Pave	NaN	IR1	Lvl	AllPub	...	0	NaN	NaN
3	4	70	RL	60.0	9550	Pave	NaN	IR1	Lvl	AllPub	...	0	NaN	NaN
4	5	60	RL	84.0	14260	Pave	NaN	IR1	Lvl	AllPub	...	0	NaN	NaN

5 rows × 81 columns

Step 3: Dataset Information

```
df.shape
df.info()
```

```
 50  HalfBath       1460 non-null   int64
 51  BedroomAbvGr   1460 non-null   int64
 52  KitchenAbvGr   1460 non-null   int64
 53  KitchenQual    1460 non-null   object
 54  TotRmsAbvGrd   1460 non-null   int64
 55  Functional     1460 non-null   object
 56  Fireplaces     1460 non-null   int64
 57  FireplaceQu    770 non-null    object
 58  GarageType     1379 non-null   object
 59  GarageYrBlt    1379 non-null   float64
 60  GarageFinish   1379 non-null   object
 61  GarageCars     1460 non-null   int64
 62  GarageArea     1460 non-null   int64
 63  GarageQual     1379 non-null   object
 64  GarageCond     1379 non-null   object
 65  PavedDrive     1460 non-null   object
 66  WoodDeckSF     1460 non-null   int64
 67  OpenPorchSF    1460 non-null   int64
 68  EnclosedPorch  1460 non-null   int64
 69  3SsnPorch      1460 non-null   int64
 70  ScreenPorch    1460 non-null   int64
 71  PoolArea       1460 non-null   int64
 72  PoolQC         7 non-null      object
 73  Fence          281 non-null    object
 74  MiscFeature    54 non-null     object
 75  MiscVal        1460 non-null   int64
 76  MoSold         1460 non-null   int64
 77  YrSold         1460 non-null   int64
 78  SaleType       1460 non-null   object
 79  SaleCondition  1460 non-null   object
 80  SalePrice      1460 non-null   int64
dtypes: float64(3), int64(35), object(43)
memory usage: 924.0+ KB
```

Step 4: Statistical Summary

```
df.describe()
```

	Id	MSSubClass	LotFrontage	LotArea	OverallQual	OverallCond	YearBuilt	YearRemodAdd	MasVnrArea	B
count	1460.000000	1460.000000	1201.000000	1460.000000	1460.000000	1460.000000	1460.000000	1460.000000	1452.000000	14
mean	730.500000	56.897260	70.049958	10516.828082	6.099315	5.575342	1971.267808	1984.865753	103.685262	4
std	421.610009	42.300571	24.284752	9981.264932	1.382997	1.112799	30.202904	20.645407	181.066207	4
min	1.000000	20.000000	21.000000	1300.000000	1.000000	1.000000	1872.000000	1950.000000	0.000000	
25%	365.750000	20.000000	59.000000	7553.500000	5.000000	5.000000	1954.000000	1967.000000	0.000000	
50%	730.500000	50.000000	69.000000	9478.500000	6.000000	5.000000	1973.000000	1994.000000	0.000000	3
75%	1095.250000	70.000000	80.000000	11601.500000	7.000000	6.000000	2000.000000	2004.000000	166.000000	7
max	1460.000000	190.000000	313.000000	215245.000000	10.000000	9.000000	2010.000000	2010.000000	1600.000000	56

8 rows × 38 columns

Step 5: Select Important Features

```
features = ['GrLivArea','BedroomAbvGr','FullBath','OverallQual']
X = df[features]
y = df['SalePrice']
```

Step 6: Train Test Split

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Step 7: Feature Scaling

```
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

Step 8: Linear Regression Model

```
lr = LinearRegression()
lr.fit(X_train, y_train)
pred_lr = lr.predict(X_test)
```

Step 9: Ridge Regression Model

```
ridge = Ridge()
ridge.fit(X_train, y_train)
pred_ridge = ridge.predict(X_test)
```

Step 10: Lasso Regression Model

```
lasso = Lasso()
lasso.fit(X_train, y_train)
pred_lasso = lasso.predict(X_test)
```

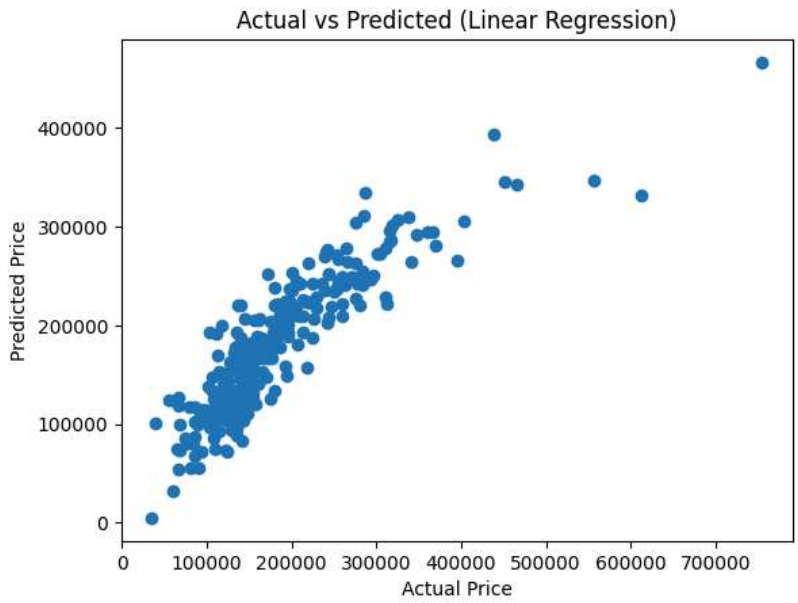
Step 11: Model Evaluation

```
print('Linear Regression RMSE:', np.sqrt(mean_squared_error(y_test, pred_lr)))
print('Linear Regression R2:', r2_score(y_test, pred_lr))

print('Ridge RMSE:', np.sqrt(mean_squared_error(y_test, pred_ridge)))
print('Ridge R2:', r2_score(y_test, pred_ridge))

print('Lasso RMSE:', np.sqrt(mean_squared_error(y_test, pred_lasso)))
print('Lasso R2:', r2_score(y_test, pred_lasso))
```

```
Linear Regression RMSE: 42813.43371836445
Linear Regression R2: 0.7610284025418284
Ridge RMSE: 42823.574649915034
Ridge R2: 0.7609151819281802
Lasso RMSE: 42813.43960899628
Lasso R2: 0.7610283367823845
```

Step 12: Actual vs Predicted Plot

```
plt.figure()
plt.scatter(y_test, pred_lr)
plt.xlabel('Actual Price')
plt.ylabel('Predicted Price')
plt.title('Actual vs Predicted (Linear Regression)')
plt.show()
```



Start coding or generate with AI.